Boosting

DS-6030 | Spring 2026

Invalid Date

Table of contents

1 Boosting **2**

2 AdaBoost **3**
2.1 AdaBoost Details 4
2.2 Adaboost Algorithm 5
2.3 LogitBoost 12

3 Gradient Boosting **13**
3.1 Gradient Descent 13
3.2 L_2 Boosting 13
3.3 GBM (Gradient Boosting Machine) 16
3.4 xgboost (Extreme Gradient Boosting) 17
3.5 CatBoost 19
3.6 LightGBM 19

4 Optimization with Taylor Expansions **21**

5 Taylor Series **22**
5.1 Optimization with Taylor 22

6 Newton Boosting and XGBoost **30**

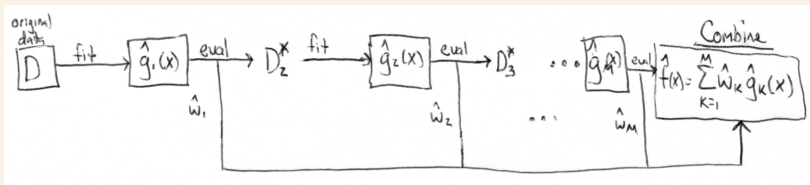
7 Appendix: L2 Tree Boosting R Code **32**

8 Appendix: xgboost and lightgbm **34**
8.1 xgboost 35
8.2 lightgbm 36
8.3 Tidymodels 37

1 Boosting

Boosting is a *sequential* ensemble method.

Boosting Sketch



- A boosting model can be written as a generic ensemble
  - M is the number of base learners
  - $\hat{a}_k$ is the weight for the kth base learner ($\hat{a}_k \geq 0$).
  - $\hat{g}_k(x)$ is the prediction from the kth base learner

$$\hat{f}_M(x) = \sum_{k=1}^{M} \hat{a}_k \, \hat{g}_k(x)$$

- The key distinction of boosting models is that the base learners are **fitted sequentially**, and the best model at stage $m+1$ is dependent on all models fit up to stage m.

$$\hat{f}_{m+1}(x) = \underset{a,\, g(x)}{\arg\min} \sum_{i=1}^{n} w_{mi} L(y_i, \hat{f}_m(x_i) + a\, g(x_i))$$

- Boosting is primarily a *bias* reducer
  - The base models are often simple/weak (low variance, but high bias) learners (like shallow trees)
- The complexity of the final model is based on i) the complexity of the base learners, ii) the number of iterations M, and iii) the magnitude of the ensemble weights a_k
  - Boosting models will overfit as the number of iterations increases
    - Early stopping is necessary
    - Less of a problem for hard classification problems with balanced data
  - Can apply *shrinkage* (making a_k smaller), to reduce complexity
- There are two main versions of boosting:
  - *Gradient Boosting*: fits the next model in the sequence $\hat{g}_k(x)$ to the (pseudo) residuals calculated from the predictions on the previous models
  - *AdaBoost*: fits the next model to sequentially *weighted* observations. The weights are proportional to the how poorly the current models predict the observation.

2 AdaBoost

AdaBoost was motivated by the idea that many *weak* leaners can be combined to produce a *strong* aggregate model.

- AdaBoost is for binary classification problems
- Trees are a popular base learner
  - *Weak* learners are usually used. For trees, this means shallow depth.
- At each iteration, the current model is evaluated.
  - The *ensemble weight* of model k is based on its performance (on all the training data)
  - The *observation weight* of observation i is increased if it is mis-classified and decreased if it is correctly classified.
  - Thus, at each iteration, those observations that are mis-classified are weighted higher and get extra attention in the next iteration.
- Because Adaboost uses hard-classifiers, it is sensitive to unbalanced data and unequal misclassification costs.
  - Because the thresholds are set at $p > .50$
  - There are, of course, ways to account for unbalance and unequal costs in the algorithm
  - An improvement to AdaBoost, *LogitBoost* explicitly attempts to estimate the class probability during each iteration which will allow easier post-fitting adjustments for unequal costs

2.1 AdaBoost Details

- Adaboost uses an outcome variable of $y \in \{-1, 1\}$
- AdaBoost implicitly uses the loss function:

$$\begin{aligned} L(y, f) &= e^{-yf} \\ &= \begin{cases} e^{-f} & y = +1 \\ e^{f} & y = -1 \end{cases} \end{aligned}$$

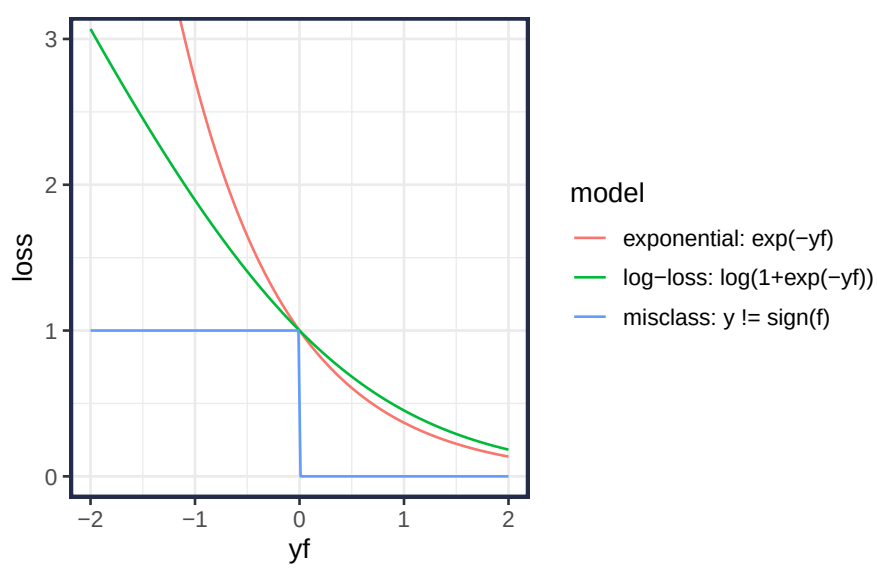
- Adaboost estimates the probability that $Y = +1$ as

$$\begin{aligned} \hat{p}(x) &= \frac{e^{\hat{f}_M(x)}}{e^{-\hat{f}_M(x)} + e^{\hat{f}_M(x)}} \\ &= \frac{e^{2\hat{f}_M(x)}}{1 + e^{2\hat{f}_M(x)}} \end{aligned}$$

where $p(x) = \Pr(Y = +1 \mid X = x)$

- And $\hat{f}(x)$ is an estimate of

$$\begin{aligned} \hat{f}_M(x) &= \frac{1}{2} \log \frac{\hat{p}(x)}{1 - \hat{p}(x)} \\ &= \frac{1}{2} \text{logit}\, \hat{p}(x) \end{aligned}$$



- Comparison with logistic regression (using log-loss / negative binomial log-likelihood)
  - $\hat{f}(x) = \text{logit}\, \hat{p}(x)$
  - $\hat{p}(x) = \frac{e^{\hat{f}_M(x)}}{1+e^{\hat{f}_M(x)}}$
  - Log-loss: $\log(1 + e^{-yf})$ (using $y \in \{-1, +1\}$)

Weighted Loss Functions (with observations weights)

Let $w_i \geq 0$ be a *weight* associated with observation i. The weighted loss for predictions $\hat{\mathbf{y}} = \hat{y}_1, \hat{y}_2, \ldots, \hat{y}_n$ is

$$L_{\mathbf{w}}(\mathbf{y}, \hat{\mathbf{y}}) = \sum_{i=1}^{n} w_i L(y_i, \hat{y}_i)$$

2.2 Adaboost Algorithm

Algorithm: AdaBoost (Discrete)

Inputs:

- $D = \{(x_i, y_i\}_{i=1}^n$, where $y_i \in \{-1, 1\}$
- Tuning parameters for base model $\hat{g}$
- Maximum number of iterations, M or other stopping criteria

Algorithm:

1. Initialize *observation weights* $w_i = 1/n$ for all i

2. For $k = 1$ to M:

   a. Fit a *classifier* $\hat{g}_k(x)$ that maps (x_i, w_i) to $\{-1, 1\}$. In other words, the classifier must make a hard classification using weighted observations.

   b. Compute the weighted mis-classification rate

   $$e_k = \frac{\sum_{i=1}^n w_i \, \mathbb{1}(y_i \neq \hat{g}_k(x_i))}{\sum_{i=1}^n w_i}$$

   Note: $0 \leq e_k \leq .5$ since model fit and evaluated on same training data.

   c. Calculate the *coefficient* for model k (*ensemble weight*)

   $$\hat{a}_k = \log\left(\frac{1 - e_k}{e_k}\right)$$

   Note: $0 \leq a_k < \infty$.

   d. Update the *observations weights*. Increase weights for observations that are mis-classified by model $\hat{g}_k$ and decrease weights for the correctly classified observations.

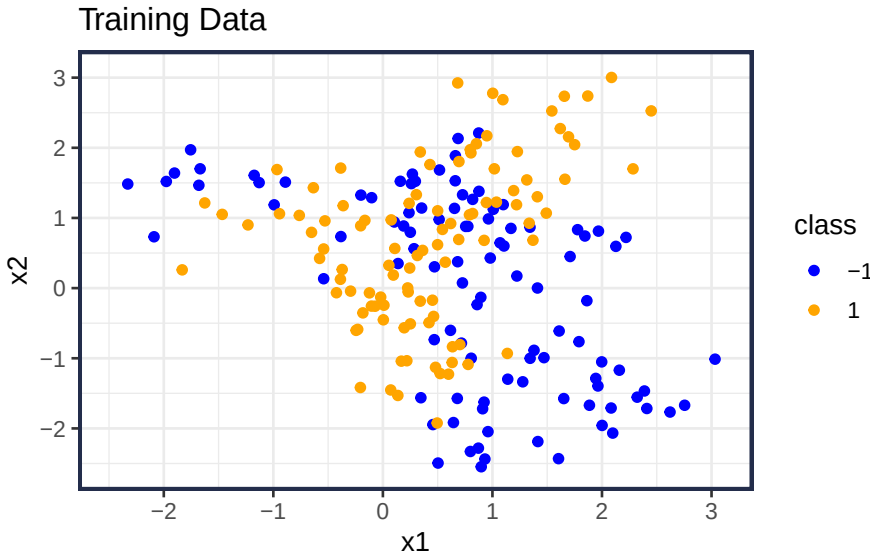
   $$\begin{aligned} \tilde{w}_i &= w_i \cdot \exp\left(\hat{a}_k \cdot \mathbb{1}(y_i \neq \hat{g}_k(x_i))\right) \\ &= \begin{cases} w_i \frac{1-e_k}{e_k} & \text{if obs } i \text{ is misclassified} \\ w_i & \text{if obs } i \text{ is correctly classified} \end{cases} \\ w_i &= \frac{\tilde{w}_i}{\sum_{j=1}^n \tilde{w}_j} \qquad (\textit{re-normalize weights}) \end{aligned}$$

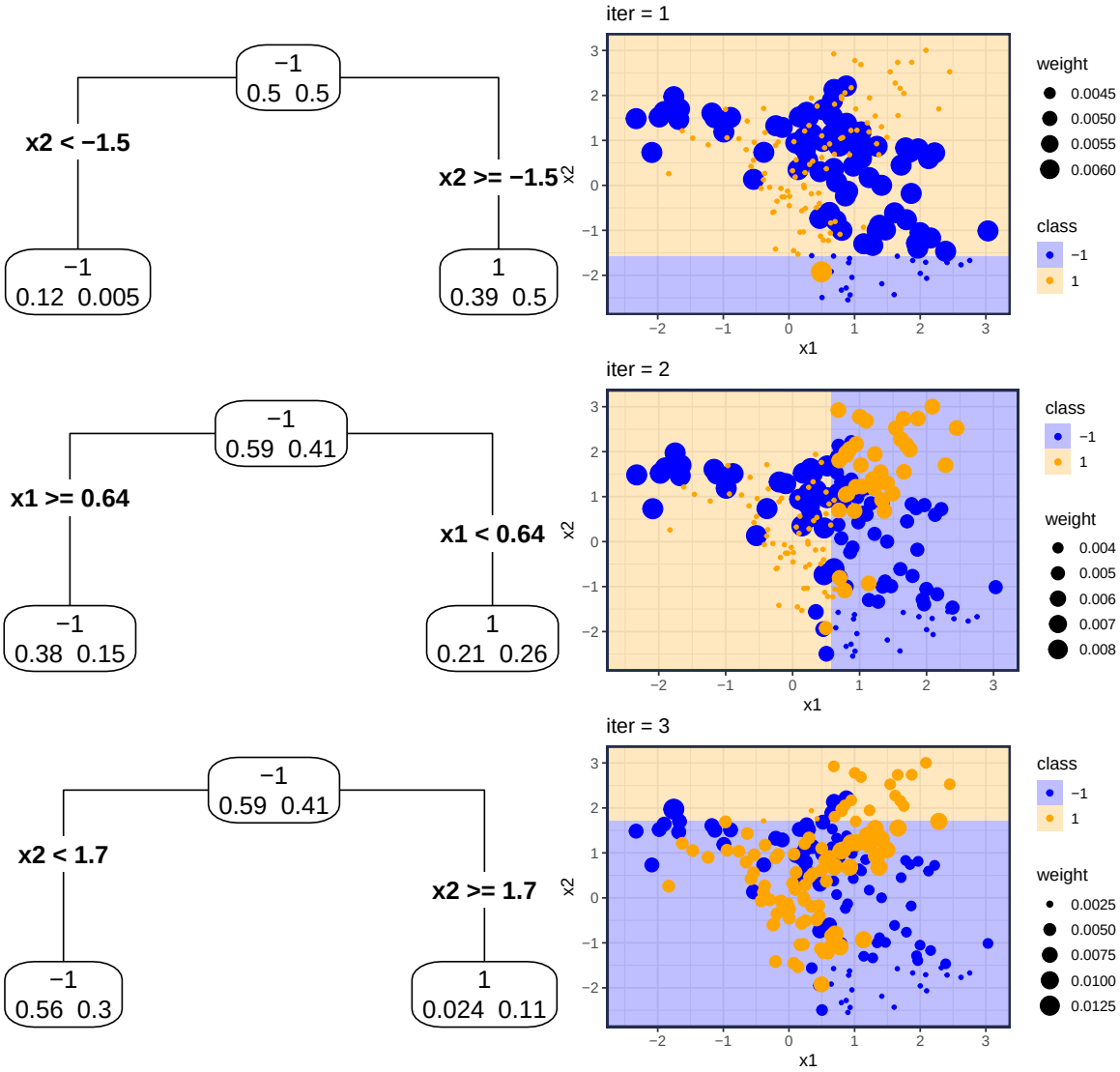
3. Output final ensemble $\hat{f}_M(x)$

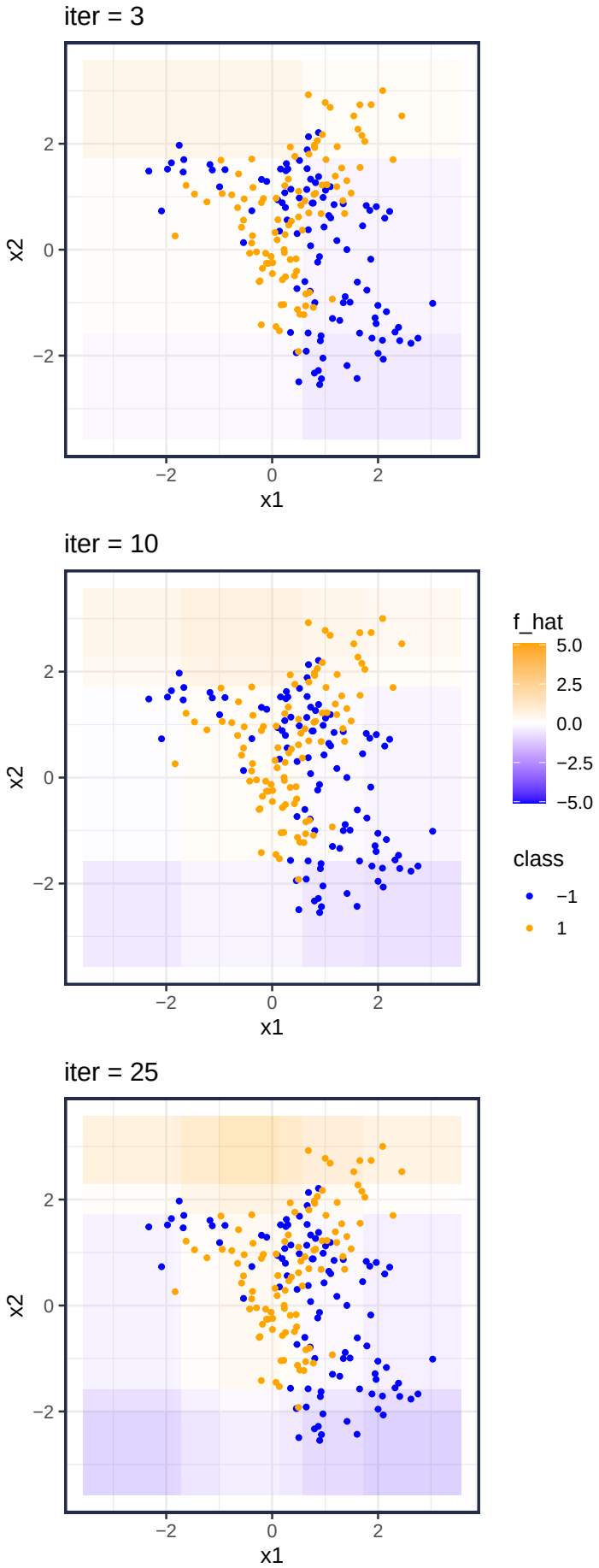
$$\hat{f}_M(x) = \sum_{k=1}^{M} \hat{a}_k \, \hat{g}_k(x)$$

- Where $\hat{f}_k(x) = \hat{a}_k \, \hat{g}_k(x)$
- Hard classification: $\hat{f}_M(x) >$ threshold
- Or remap to a probability $\hat{p}(x) = \frac{e^{2\hat{f}(x)}}{1+e^{2\hat{f}(x)}}$ for thresholding

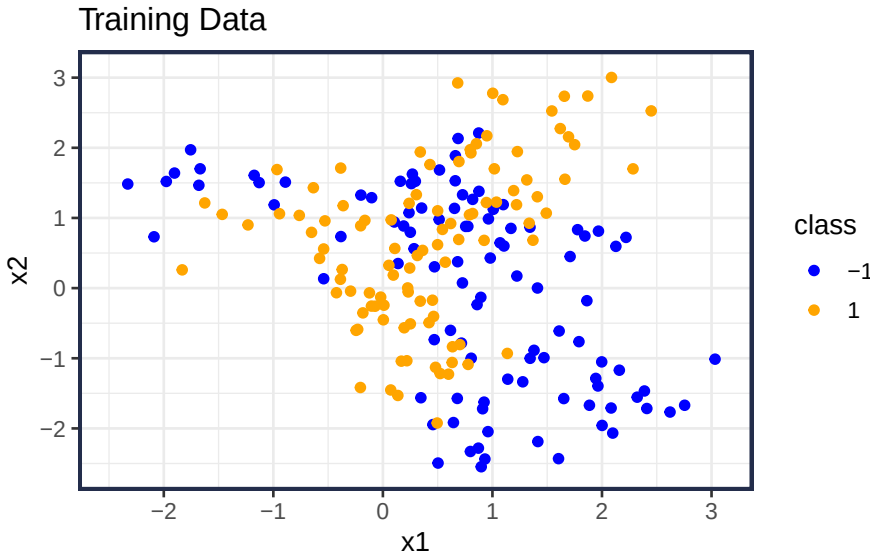
2.2.1 Illustration with Stumps (depth = 1, n.nodes=2)

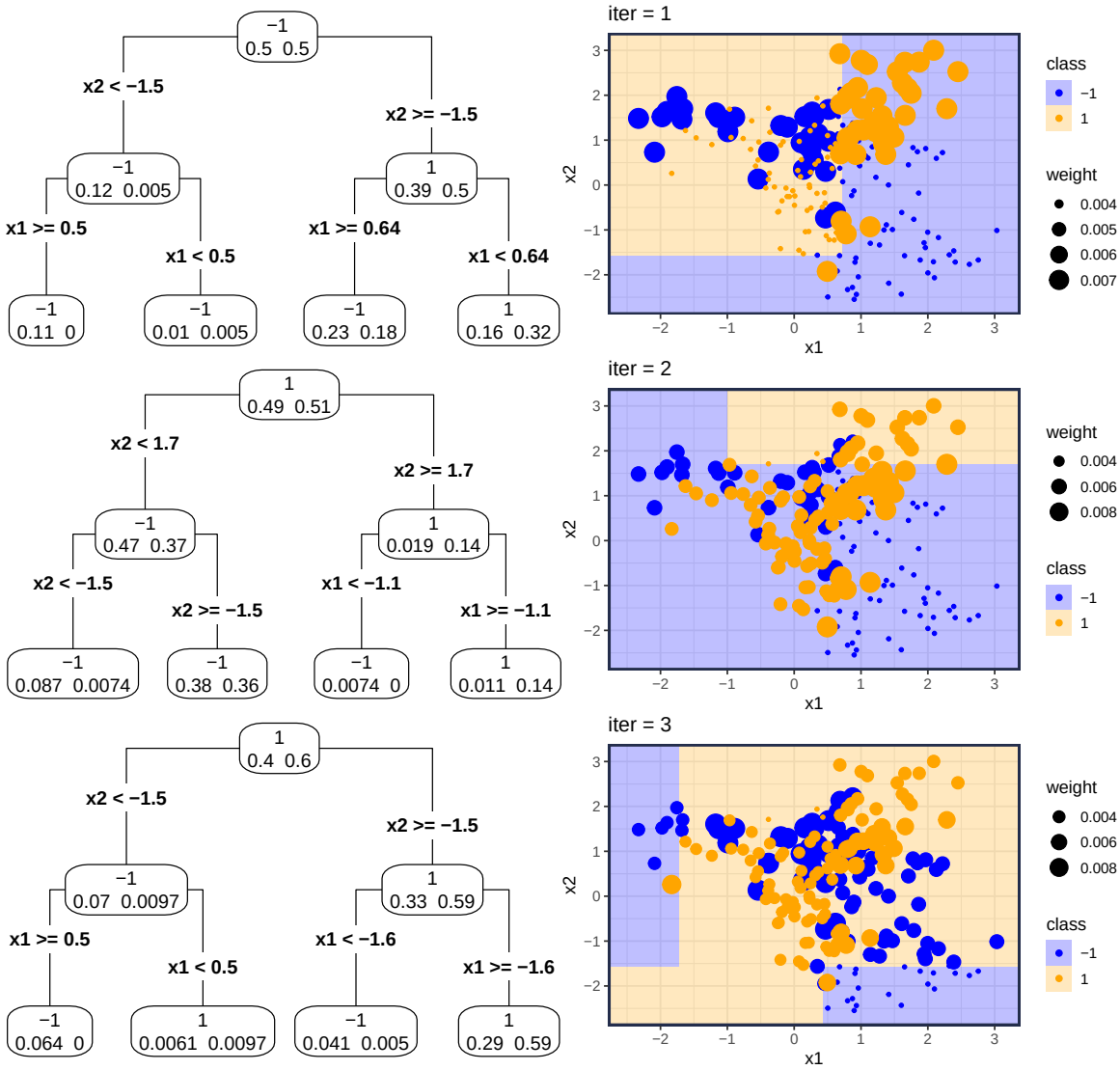


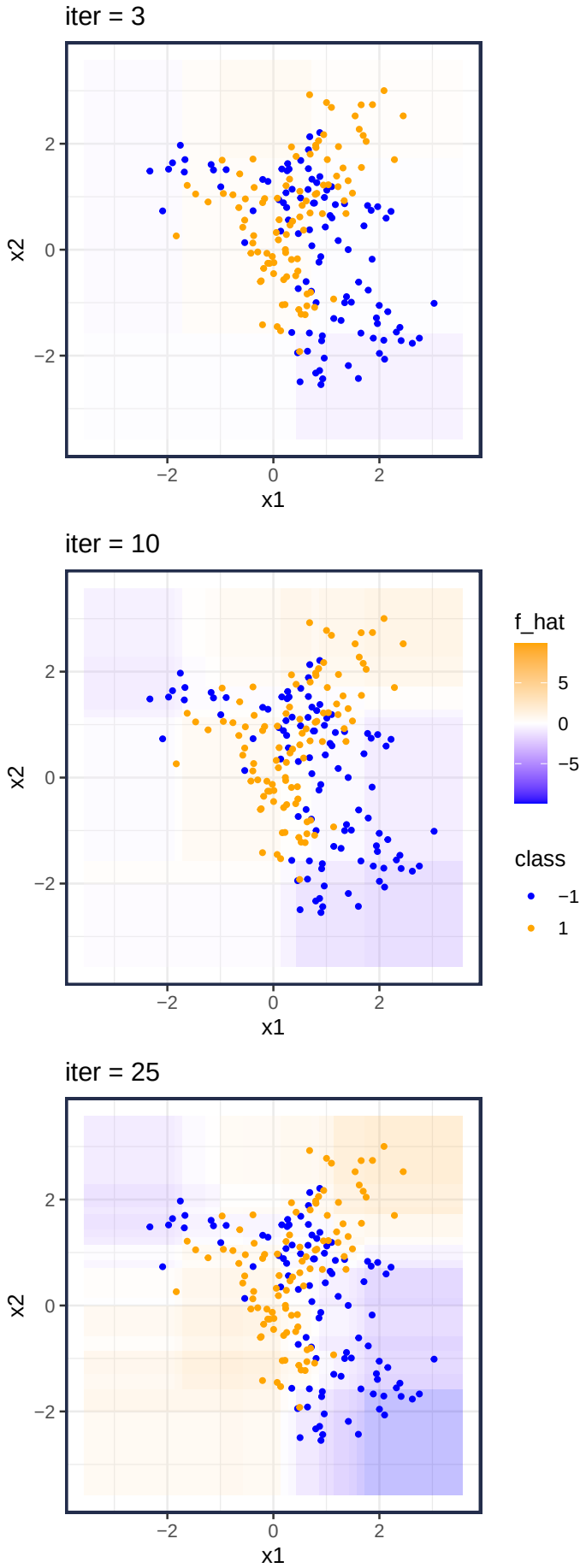


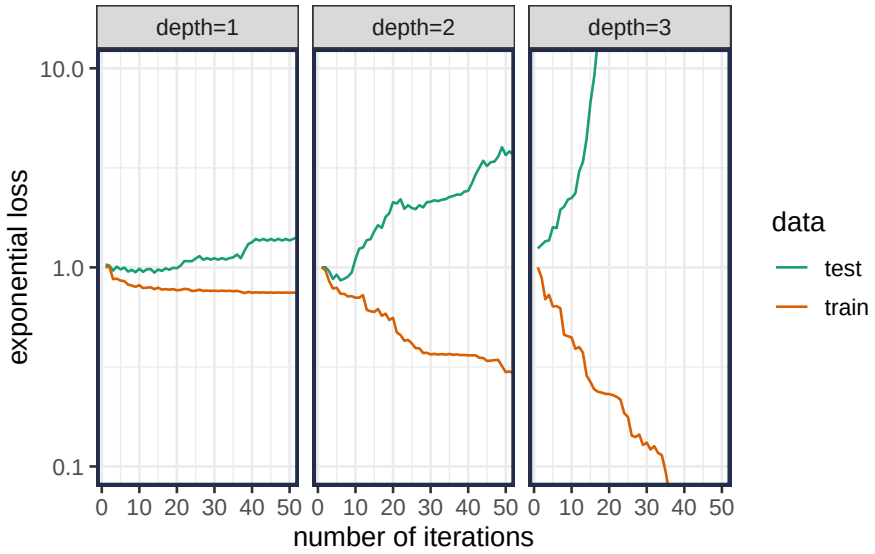

iter = 3
iter = 10
iter = 25
x1
x2
−2
0
2
f_hat
5.0
2.5
0.0
−2.5
−5.0
class
−1
1

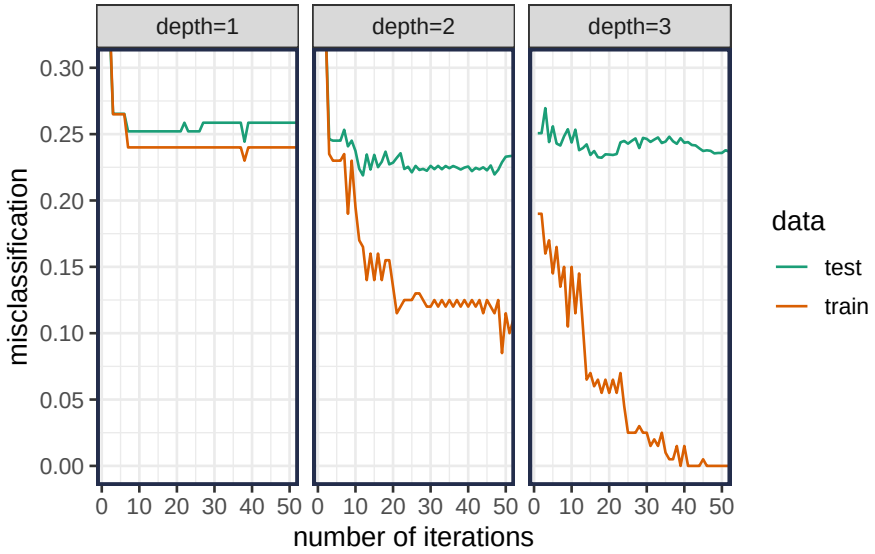
2.2.2 Illustration with depth = 2, n.nodes=4






iter = 3
iter = 10
iter = 25
x1
x2
−2
0
2
f_hat
5
0
−5
class
−1
1


depth=1
depth=2
depth=3
exponential loss
10.0
1.0
0.1
0 10 20 30 40 50
number of iterations
data
test
train


depth=1
depth=2
depth=3
misclassification
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0 10 20 30 40 50
number of iterations
data
test
train

2.3 LogitBoost

- See Friedman, J., Hastie, T., and Tibshirani, R. (2000). Additive Logistic Regression: A statistical view of boosting. Annals of Statistics, 28(2), 337-374. for the details of model variations

Algorithm: LogitBoost

Inputs:

- $D = \{(x_i, y_i\}_{i=1}^n$, where $y_i \in \{-1, 1\}$
- Tuning parameters for base model $\hat{g}$
- Maximum number of iterations, M
- Let $y_i^* = (y+1)/2 \in \{0, 1\}$

Algorithm:

1. Initialize *observation weights* $w_i = 1/n$ for all i and $f_0(x) = 0$

2. For $k = 1$ to M:

   a. Like in newton-raphson for logistic regression, calculate the working response and weights for all observations

$$z_i = \frac{y_i^* - p_i}{p_i(1-p_i)}$$
$$w_i = p_i(1-p_i)$$

   b. Fit a model $\hat{g}_k(x)$ with weighted least squares that estimates z_i using features x_i and weights w_i.

   c. Update $\hat{f}_k(x) = \hat{f}_{k-1}(x) + \hat{g}_k(x)/2$ and $p_i = e^{\hat{f}_k(x)}/(e^{\hat{f}_k(x)} + e^{-\hat{f}_k(x)})$

3. Output final ensemble $\hat{f}_M(x) \in \mathbb{R}$

$$\hat{f}_M(x) = \sum_{k=1}^{M} \frac{1}{2}\, \hat{g}_k(x)$$

   - Where $\hat{f}_k(x) = \frac{1}{2}\, \hat{g}_k(x)$.
   - Hard classification: $\hat{f}_M(x) > 0$
   - Or remap to a probability $\hat{p}(x) = \frac{e^{2f}}{1+e^{2f}}$ for thresholding

3 Gradient Boosting

The boosting model:

$$\hat{f}_M(x) = \sum_{k=1}^{M} \hat{a}_k \, \hat{g}_k(x)$$

Sequential Fitting:

$$\hat{f}_{k+1}(x) = \underset{a,\, g(x)}{\arg\min} \sum_{i=1}^{n} L(y_i, \hat{f}_k(x_i) + a\, g(x_i))$$

The concept of gradient boosting is to sequentially re-fit to the negative (functional) gradients of the loss function (think of the negative gradients as a type of residual; *pseudo residual*).

- The same structure can be used for many different loss functions
  - it works the same for regression and classification
  - survival analysis, ranking, etc.

3.1 Gradient Descent

- Our objective is to find the model (or model parameters) that minimize the loss function
- From any starting point, we can move toward the optimum using *gradient descent*:

$$f_{k+1} = f_k - \nu_k \, L'(f_k)$$

  - $\nu_k > 0$ is the step-size
  - $L'(f_k)$ is the functional derivative of the loss with respect to the model f_k
- Boosting fits models sequentially:

$$\hat{f}_{k+1}(x) = \hat{f}_k(x) + \hat{a}_k \, \hat{g}_k(x)$$

- So we see a parallel; each boosting model $\hat{g}_k(x)$ can be viewed as estimating the *negative derivative* of the loss function.

3.2 L_2 Boosting

L_2 boosting is based on the the squared error loss function

$$L(y_i, \hat{f}(x_i)) = \frac{1}{2}(y_i - \hat{f}(x_i))^2$$

- The *negative gradients* are

$$\begin{aligned} r_i &= \left[-\frac{\partial L(y_i, f_i)}{\partial f_i} \right]_{f_i = \hat{f}(x_i)} \\ &= y_i - \hat{f}(x_i) \end{aligned}$$

- L2 Boosting is simply re-fitting to the residuals.

Algorithm: L_2 Boosting

1. Initialize $\hat{f}_0(x) = \bar{y}$

2. For $k = 1$ to M:

   a. Calculate residuals $r_i = y_i - \hat{f}_{k-1}(x_i)$ for all i

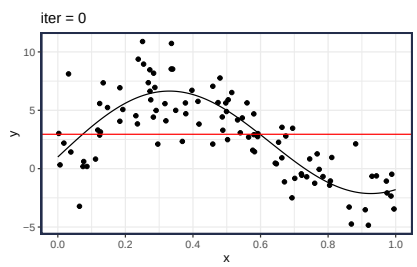
   b. Fit a base learner (e.g., regression trees) to the residuals $\{(x_i, r_i)\}_{i=1}^n$ to get the model $\hat{g}_k(x)$

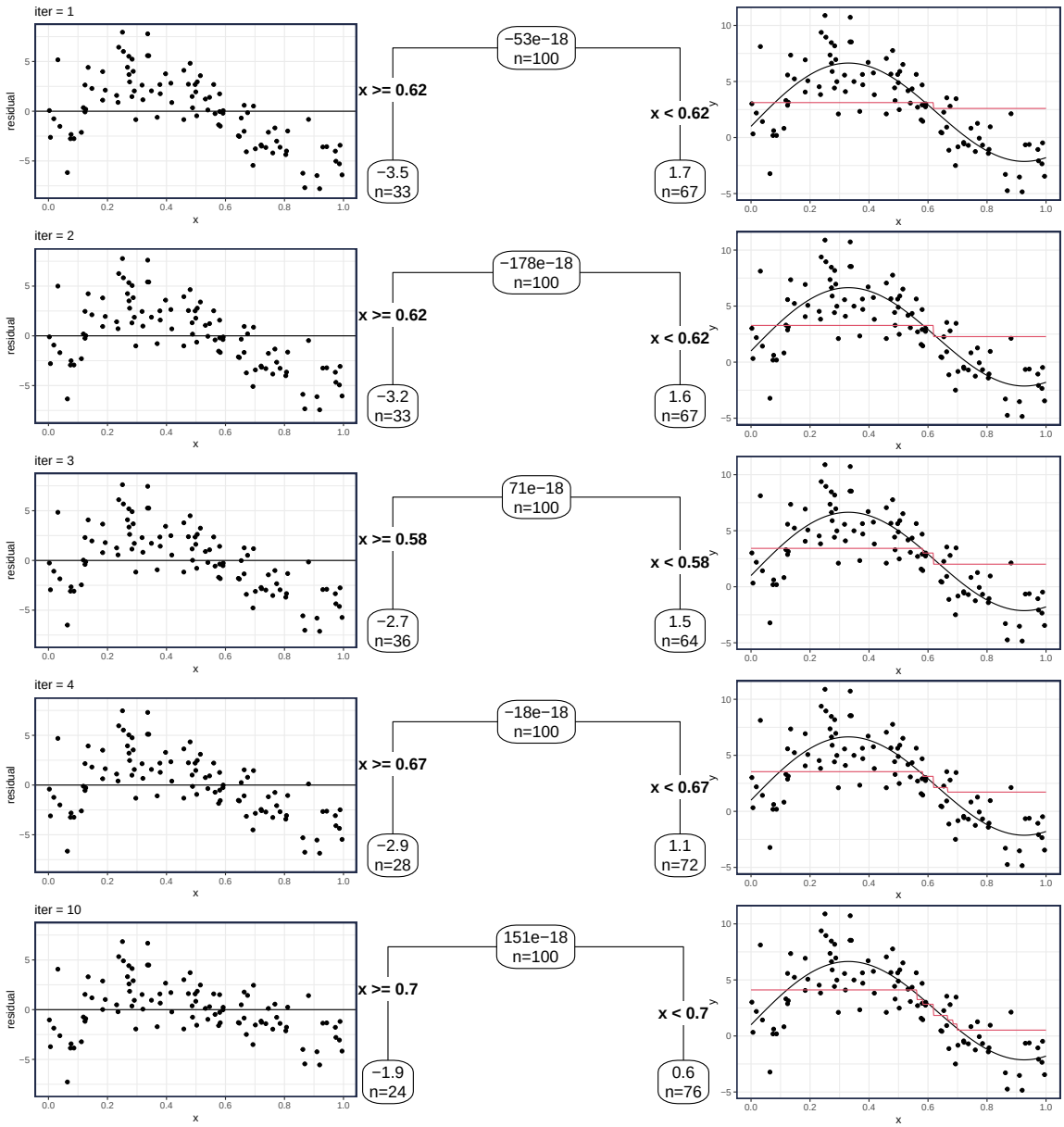
   c. Update the overall model $\hat{f}_k(x) = f_{k-1}(x) + \nu \hat{g}_k(x)$
      - $0 \leq \nu \leq 1$ is the step-size (*shrinkage*)

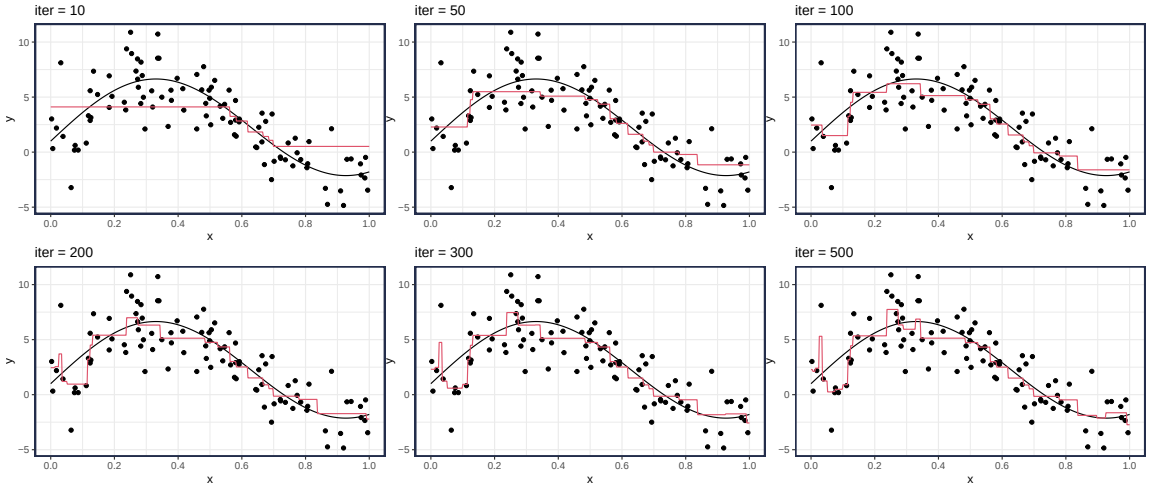
3. Final model is $\hat{f}_M(x) = \bar{y} + \sum_{k=1}^M \nu \hat{g}_k(x)$

- Like AdaBoost, emphasis is given to observations that are predicted poorly (large residuals)

3.2.1 Illustration using stumps (depth=1, n.nodes=2, $\nu = .1$)




iter = 1
residual
x
−53e−18
n=100
x >= 0.62
x < 0.62
−3.5
n=33
1.7
n=67
y
iter = 2
−178e−18
n=100
−3.2
n=33
1.6
n=67
iter = 3
71e−18
n=100
x >= 0.58
x < 0.58
−2.7
n=36
1.5
n=64
iter = 4
−18e−18
n=100
x >= 0.67
x < 0.67
−2.9
n=28
1.1
n=72
iter = 10
151e−18
n=100
x >= 0.7
x < 0.7
−1.9
n=24
0.6
n=76


iter = 10
iter = 50
iter = 100
iter = 200
iter = 300
iter = 500
y
x

3.3 GBM (Gradient Boosting Machine)

- R package `gbm`
- GBM Documentation
- GBM is a first order approach. It does not consider Hessian.

3.3.1 Model/Tree Tuning Parameters

- Tree depth (`interaction.depth`)
  - Grows trees to a depth specified by `interaction.depth` (unless there are not enough observations in the terminal nodes)
- Minimum number of observations allowed in the terminal nodes (`n.minobsinnode`)
- Sub-sampling (`bag.fraction`)
  - *Stochastic Gradient Boosting*
  - Sample (without replacement) at each iteration
- Loss Function (`distribution`)
  - The loss function is determined by the `distribution` argument
  - Use `distribution="gaussian"` for squared error
  - Other options are: `bernoulli` (for logistic regression), `poisson` (for Poisson regression), `pairwise` (for ranking/LambdaMart), `adaboost` (for the adaboost exponential loss), etc.

3.3.2 Boosting Tuning Parameters

- Number of iterations/trees (`n.trees`)
  - Need to tune
- Shrinkage parameter (`shrinkage`)
  - Set small, but the smaller the `shrinkage`, the more iterations/trees need to be used
  - "Ranges from 0.001 to 0.100 usually work"
- Cross-validation (`cv.folds`)
  - `gbm` has a built in cross-validation
  - no way to manually set the folds

3.3.3 Computational Settings

- Number of Cores (`n.cores`)
  - Only used when cross-validation is implemented

3.4 xgboost (Extreme Gradient Boosting)

- R package `xgboost`
- xgboost Documentation
- xgboost Model
- xbgoost Paper

3.4.1 Model/Tree Tuning Parameters

- Different base leaners (`booster`)
  - `gbtree` is a tree
  - `gblinear` creates a (generalized) liner model (forward stagewise linear model)
- Tree building (`tree_method`)
  - To speed up the fitting, only consider making splits at certain quantiles of the input vector (rather than considering every unique value)
- Sub-sampling (`subsample`)
  - *Stochastic Gradient Boosting*
  - Sample (without replacement) at each iteration
- Feature sampling (`colsample_bytree`, `colsample_bylevel`, `colsample_bynode`)
  - Like used in Random Forest, the features/columns are subsampled
  - Can use a subsample of features for each tree, level, or node

Model Complexity Parameters

- Tree depth (`max_depth`)
  - Grows trees to a depth specified by `max_depth` (unless there are not enough observations in the terminal nodes)
  - Trees may not reach `max_depth` if the `gamma` or `min_child_weight` arguments are set.
- Minimum number of observations (or sum of weights) allowed in the terminal nodes (`min_child_weight`)
- Pruning (`gamma` or `min_split_loss`)
  - Minimum loss reduction required to make a further partition on a leaf node of the tree
  - The larger gamma is, the more conservative the algorithm will be
- ElasticNet type penalty (`lambda` and `alpha`)
  - `lambda` is an L_2 penalty
  - `alpha` is an L_1 penalty

Note

- Recall that trees model the outcome as a *constant* in each region

$$\hat{f}_T(x) = \sum_{m=1}^{M} \hat{c}_m \, \mathbb{1}(x \in \hat{R}_m)$$

- Cost-complexity pruning found the optimal tree as the one that minimized the penalized loss objective function:

$$C_\gamma(T) = \sum_{m=1}^{|T|} \text{Loss}(T) + \gamma |T|$$

- XGBoost selects a tree at each iteration using the following penalized loss:

$$C_{\gamma,\lambda,\alpha}(T) = \sum_{m=1}^{|T|} \text{Loss}(T) + \gamma |T| + \frac{\lambda}{2} \sum_{m=1}^{|T|} \hat{c}_m^2 + \alpha \sum_{m=1}^{|T|} |\hat{c}_m|$$

- Loss Function (`objective`)
  - The loss function is determined by the `objective` argument
  - Use `reg:squarederror` for squared error
  - Other options are: `reg:logistic` or `binary:logistic` (for logistic regression), `count:poisson` (for Poisson regression), `rank:pairwise` (for ranking/LambdaMart), etc.

3.4.2 Boosting Tuning Parameters

- Shrinkage parameter (`eta` or `learning_rate`)
  - Set small, but the smaller the `eta`, the more iterations/trees need to be used
- Number of iterations/trees (`num_rounds`)
- Cross-validation (`xgb.cv`)
  - `xgboost` has a built in cross-validation
  - It is possible to manually set the folds

3.4.3 Computational Settings

- Number of Threads (`nthread`)
- GPU Support (https://xgboost.readthedocs.io/en/latest/gpu/index.html)
  - Used for finding tree split points and evaluating/calculating the loss function

3.5 CatBoost

- R package: (https://github.com/catboost/catboost/tree/master/catboost/R-package)
- CatBoost Documentation
- Model/Tree Tuning Parameters:

- Boosting Tuning Parameters:

3.6 LightGBM

- R Package: https://github.com/microsoft/LightGBM/tree/master/R-package
- LightGBM Documentation
- Model/Tree Tuning Parameters:

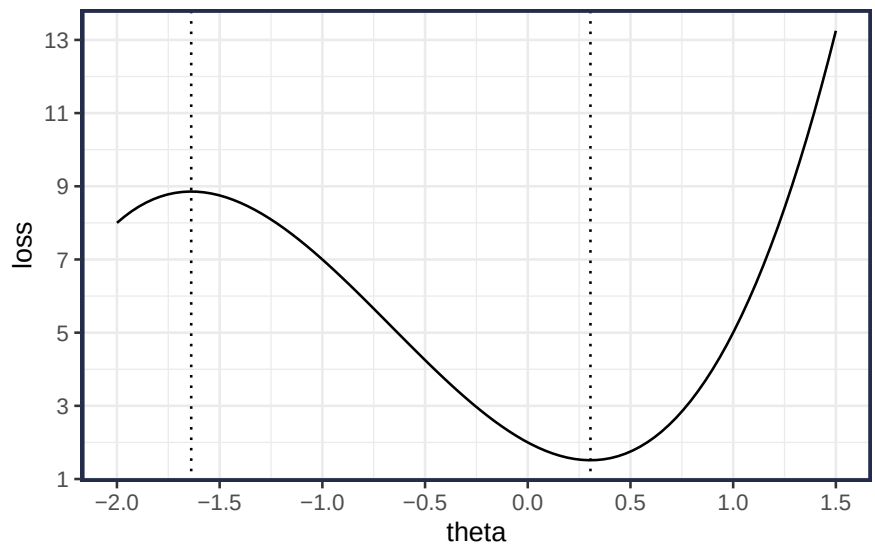
- Boosting Tuning Parameters:

4 Optimization with Taylor Expansions

Consider the problem of finding the parameter that minimizes the function

$$l(\theta) = 2 - 3\theta + 4\theta^2 + 2\theta^3$$

in the range of $\theta \in [-2, 1.5]$.



There are several options including:

- Grid search
  a. Evaluate at a grid of θ values.
  b. Choose the one that minimizes the loss.
- Sequential Optimization / Design of Experiments / Response Surface / Surrogate Model / Bayesian Optimization
  a. Select initial evaluation points according to an experimental design.
  b. Fit a model to the loss (e.g., quadratic, Gaussian Process).
  c. Use the model to suggest a new evaluation points.
  d. Repeat until desired convergence.
- Gradient Descent
  – sequential optimization using 1st order approximations.
- Newton's Method
  – sequential optimization using 2nd order approximations.

We will specifically focus on the basics of the last two.

5 Taylor Series

A Taylor Series (or Taylor Expansion) of a function approximates the function around a given input value. For example, the nth order Taylor approximation to our function l around a specific input θ

$$\begin{aligned} l(\theta+\epsilon) &\approx l(\theta) + \frac{l^{(1)}(\theta)}{1!}\epsilon + \frac{l^{(2)}(\theta)}{2!}\epsilon^2 + \ldots + \frac{l^{(n)}(\theta)}{n!}\epsilon^n \\ &= \sum_{j=0}^{n} \frac{l^{(j)}(\theta)}{j!}\epsilon^j \end{aligned}$$

Where $\epsilon \in \mathbb{R}$ is a step (direction and size) and $l^{(j)}(\theta) = d^j l(\theta)/(d\theta)^j$ is the jth derivative of the function evaluated at input θ.

For our particular function we have

$$\begin{aligned} l(\theta) &= 2 - 3\theta + 4\theta^2 + 2\theta^3 \\ l^{(1)}(\theta) &= g(\theta) = -3 + 8\theta + 6\theta^2 \\ l^{(2)}(\theta) &= h(\theta) = 8 + 12\theta \\ l^{(3)}(\theta) &= 12 \\ l^{(4)}(\theta) &= 0 \end{aligned}$$

5.1 Optimization with Taylor

Let's take a guess and start with $\theta_1 = 1$. We evaluate the function and find $L(\theta = 1) = 5$. Where should we try next?

Let's consider the Taylor approximations.

- The 1st order (linear) approximation is:

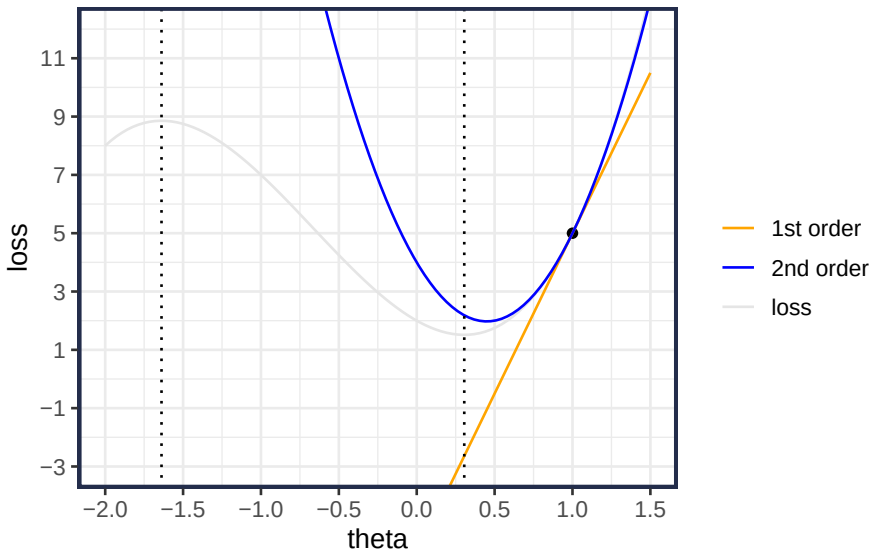
$$\begin{aligned} l(1+\epsilon) &\approx l(1) + g(1)\epsilon \\ &= 5 + 11\epsilon \end{aligned}$$

- The 2nd order (quadratic) approximation is:

$$\begin{aligned} l(1+\epsilon) &\approx l(1) + g(1)\epsilon + (h(1)/2)\epsilon^2 \\ &= 5 + 11\epsilon + 10\epsilon^2 \end{aligned}$$

Note

Notice that we evaluate the derivatives at $\theta = 1$, not at $1 + \epsilon$! These are constants, not a function of ϵ.



Which input should we try next?

5.1.1 Gradient Descent

Gradient descent is an optimization algorithm based on the 1st order (linear) approximation. The basic idea is to take a step in the direction of the negative gradient. That is, the next input to try is:

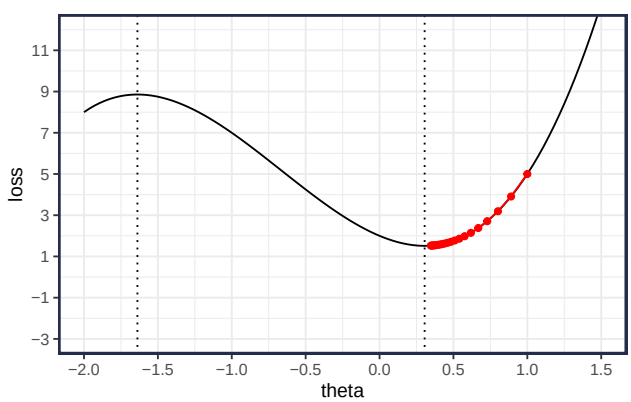
$$\theta_2 = \theta_1 - \nu g(\theta_1)$$

where $g(\theta_1) = l^{(1)}(\theta_1)$ is the *gradient* and ν is the *step size*. Recall our notation $\theta_2 = \theta_1 + \epsilon$ implies that gradient descent suggests the step $\epsilon^* = -\nu g(\theta_1)$.

The step size parameter ν is important. If it's too large, you can overshoot the minimum; if it's too small, it will take you too many iterations to converge. There are lots of approaches at optimizing the step size (most make step size smaller over the iterations), but these issues are beyond what we can cram into our class.

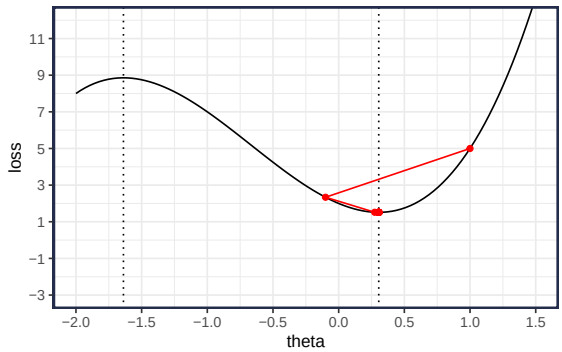
Gradient descent is an iterative approach; keep taking steps proportional to the negative gradient until the loss doesn't decrease very much (note danger of over-stepping) or the maximum number of iterations are reached.

5.1.1.1 Using $\nu = 0.01$



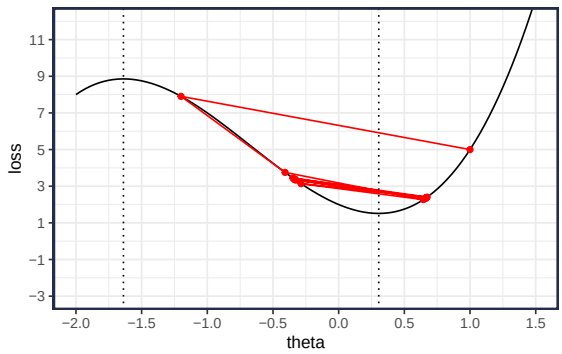
iter	nu	theta	gradient	epsilon	loss	delta
0	0.01	1.00	11.00	NA	5.00	NA
1	0.01	0.89	8.87	-0.11	3.91	1.09
2	0.01	0.80	7.26	-0.09	3.19	0.72
3	0.01	0.73	6.01	-0.07	2.71	0.48
4	0.01	0.67	5.03	-0.06	2.38	0.33
5	0.01	0.62	4.24	-0.05	2.15	0.23
6	0.01	0.58	3.60	-0.04	1.98	0.17
7	0.01	0.54	3.07	-0.04	1.86	0.12
8	0.01	0.51	2.63	-0.03	1.77	0.09
9	0.01	0.48	2.26	-0.03	1.71	0.06
10	0.01	0.46	1.95	-0.02	1.66	0.05
11	0.01	0.44	1.69	-0.02	1.63	0.04
12	0.01	0.42	1.47	-0.02	1.60	0.03
13	0.01	0.41	1.28	-0.01	1.58	0.02
14	0.01	0.40	1.11	-0.01	1.56	0.02
15	0.01	0.39	0.97	-0.01	1.55	0.01
16	0.01	0.38	0.85	-0.01	1.54	0.01
17	0.01	0.37	0.74	-0.01	1.54	0.01
18	0.01	0.36	0.65	-0.01	1.53	0.01
19	0.01	0.35	0.57	-0.01	1.53	0.00
20	0.01	0.35	0.50	-0.01	1.52	0.00

5.1.1.2 Using $\nu = 0.10$



iter	nu	theta	gradient	epsilon	loss	delta
0	0.1	1.00	11.00	NA	5.00	NA
1	0.1	-0.10	-3.74	-1.10	2.34	2.66
2	0.1	0.27	-0.36	0.37	1.52	0.82
3	0.1	0.31	0.05	0.04	1.51	0.01
4	0.1	0.30	-0.01	-0.01	1.51	0.00
5	0.1	0.31	0.00	0.00	1.51	0.00
6	0.1	0.31	0.00	0.00	1.51	0.00
7	0.1	0.31	0.00	0.00	1.51	0.00
8	0.1	0.31	0.00	0.00	1.51	0.00
9	0.1	0.31	0.00	0.00	1.51	0.00
10	0.1	0.31	0.00	0.00	1.51	0.00
11	0.1	0.31	0.00	0.00	1.51	0.00
12	0.1	0.31	0.00	0.00	1.51	0.00
13	0.1	0.31	0.00	0.00	1.51	0.00
14	0.1	0.31	0.00	0.00	1.51	0.00
15	0.1	0.31	0.00	0.00	1.51	0.00
16	0.1	0.31	0.00	0.00	1.51	0.00
17	0.1	0.31	0.00	0.00	1.51	0.00
18	0.1	0.31	0.00	0.00	1.51	0.00
19	0.1	0.31	0.00	0.00	1.51	0.00
20	0.1	0.31	0.00	0.00	1.51	0.00

5.1.1.3 Using $\nu = 0.20$



iter	nu	theta	gradient	epsilon	loss	delta
0	0.2	1.00	11.00	NA	5.00	NA
1	0.2	-1.20	-3.96	-2.20	7.90	-2.90
2	0.2	-0.41	-5.27	0.79	3.75	4.15
3	0.2	0.65	4.66	1.05	2.27	1.49
4	0.2	-0.29	-4.80	-0.93	3.14	-0.87
5	0.2	0.67	5.11	0.96	2.40	0.74
6	0.2	-0.35	-5.06	-1.02	3.45	-1.04
7	0.2	0.66	4.95	1.01	2.35	1.09
8	0.2	-0.33	-4.97	-0.99	3.33	-0.98
9	0.2	0.67	5.02	0.99	2.38	0.96
10	0.2	-0.34	-5.01	-1.00	3.39	-1.01
11	0.2	0.67	4.99	1.00	2.37	1.02
12	0.2	-0.33	-4.99	-1.00	3.36	-1.00
13	0.2	0.67	5.00	1.00	2.37	0.99
14	0.2	-0.33	-5.00	-1.00	3.37	-1.00
15	0.2	0.67	5.00	1.00	2.37	1.00
16	0.2	-0.33	-5.00	-1.00	3.37	-1.00
17	0.2	0.67	5.00	1.00	2.37	1.00
18	0.2	-0.33	-5.00	-1.00	3.37	-1.00
19	0.2	0.67	5.00	1.00	2.37	1.00
20	0.2	-0.33	-5.00	-1.00	3.37	-1.00

5.1.2 Newton's Method

Newton's method is an optimization algorithm based on the 2nd order (quadratic) approximation. Recall the Taylor's second order approximation of the loss function:

$$l(\theta + \epsilon) = l(\theta) + g(\theta)\epsilon + (h(\theta)/2)\epsilon^2$$

where $g(\theta)$ is the *gradient* and $h(\theta)$ is the *hessian*.

If we want to minimize the loss, we can write

$$\begin{aligned}\epsilon^* &= \arg\min_{\epsilon}\ l(\theta + \epsilon) \\ &\approx \arg\min_{\epsilon}\ l(\theta) + g(\theta)\epsilon + (h(\theta)/2)\epsilon^2\end{aligned}$$

Take the derivative with respect to ϵ, set to 0 and solve for ϵ:

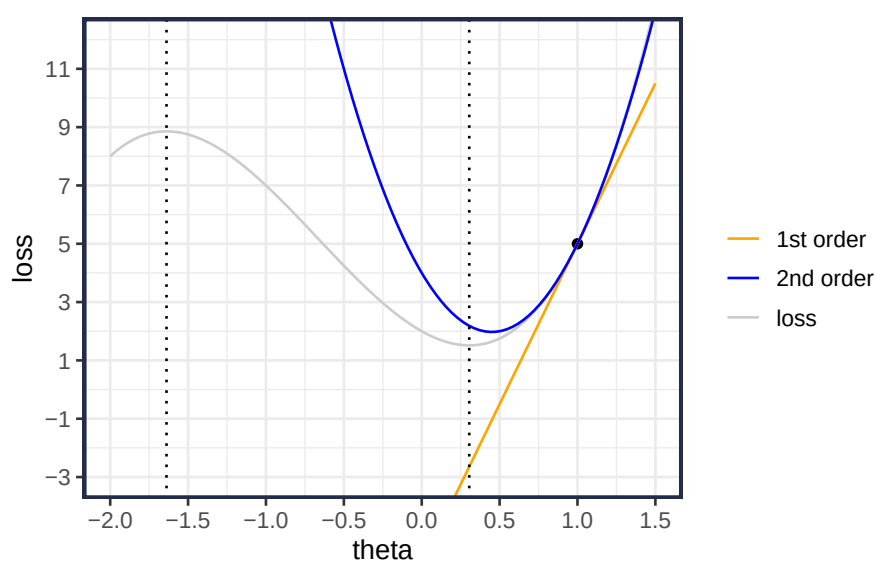
$$\begin{aligned}\frac{dl(\theta+\epsilon)}{d\epsilon} &= g(\theta) + h(\theta)\epsilon = 0 \\ \epsilon^* &= -\frac{g(\theta)}{h(\theta)}\end{aligned}$$

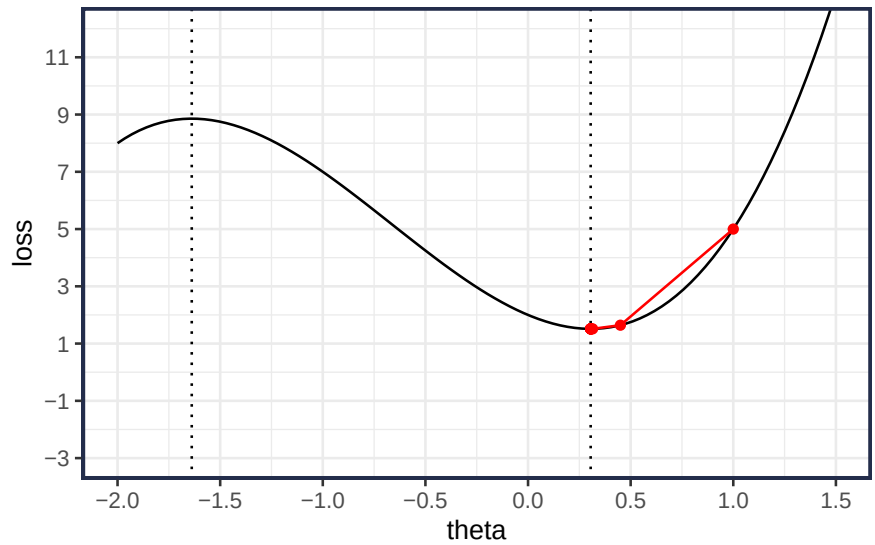
This suggests using

$$\begin{aligned}\theta_2 &= \theta_1 - \frac{g(\theta_1)}{h(\theta_1)} \\ &= \theta_1 - \nu g(\theta_1)\end{aligned}$$

where the step size $\nu = \frac{1}{h(\theta_1)}$ is given by one over the hessian.

Newton's method is also an iterative approach; keep iterating until the loss doesn't decrease very much (note danger of over-stepping) or the maximum number of iterations are reached. Note, it is still possible to overshoot, so adding shrinkage (i.e., force ν closer to 0) is sometimes done with Newton's method.





iter	nu	theta	gradient	hessian	epsilon	loss	delta
0	NA	1.00	11.00	20.00	NA	5.00	NA
1	0.05	0.45	1.81	13.40	-0.55	1.64	3.36
2	0.07	0.31	0.11	11.77	-0.14	1.51	0.13
3	0.08	0.31	0.00	11.66	-0.01	1.51	0.00
4	0.09	0.31	0.00	11.66	0.00	1.51	0.00
5	0.09	0.31	0.00	11.66	0.00	1.51	0.00
6	0.09	0.31	0.00	11.66	0.00	1.51	0.00
7	0.09	0.31	0.00	11.66	0.00	1.51	0.00
8	0.09	0.31	0.00	11.66	0.00	1.51	0.00
9	0.09	0.31	0.00	11.66	0.00	1.51	0.00
10	0.09	0.31	0.00	11.66	0.00	1.51	0.00

6 Newton Boosting and XGBoost

A boosting model can be written:

$$\hat{F}_M(x) = \sum_{m=1}^{M} \hat{a}_m \hat{f}_m(x)$$

where $\hat{a}_m$ is the weight assigned to model $\hat{f}_m(x)$. They are fit sequentially such that at iteration m the goal is to choose $\hat{f}_m(x)$ to minimize the loss:

$$\hat{f}_m(x) = \arg\min_f \sum_{i=1}^{n} l(y_i, \hat{F}_{m-1}(x_i) + f(x_i))$$

Let's re-write the loss, at iteration m, for observation i

$$l(F_i + f_i) = l(y_i, \hat{F}_{m-1}(x_i) + f(x_i))$$

and approximate it with a 2nd order (quadratic) Taylor expansion:

$$\begin{aligned} l(F_i + f_i) &\approx l(F_i) + g_i f(x_i) + \frac{1}{2} h_i f^2(x_i) \\ &= \frac{1}{2} h_i \left(-g_i/h_i - f(x_i)\right)^2 - \left[g_i^2/(2h_i) + l(F_i)\right] \end{aligned}$$

where $g_i = \frac{\partial l(F_i)}{\partial F_i}$ is the *gradient* and $h_i = \frac{\partial^2 l(F_i)}{\partial F_i^2}$ is the *hessian*.

Gradient and Hessian for common loss functions

- **Squared Error Loss**

$$\begin{aligned} \ell(y_i, F_i) &= \frac{1}{2}(y_i - F_i)^2 \\ g_i &= -(y_i - F_i) \\ h_i &= 1 \end{aligned}$$

- **Log Loss (negative Bernoulli log-likelihood, cross-entropy)**

$$\begin{aligned} y_i &\in \{0, 1\} \\ p_i &= \frac{e^{F_i}}{1 + e^{F_i}} \\ \ell(y_i, F_i) &= -\left(y_i \log p_i + (1 - y_i) \log(1 - p_i)\right) \\ &= -\left(y_i F_i - \log(1 + e^{F_i})\right) \\ g_i &= -(y_i - p_i) \\ h_i &= p_i(1 - p_i) \end{aligned}$$

Using this approximation, we can consider our optimization problem at iteration m as finding the

model such that

$$\begin{aligned}
\hat{f}_m(x) &= \arg\min_f \sum_{i=1}^n l(y_i, \hat{F}_{m-1}(x_i) + f(x_i)) \\
&\approx \arg\min_f \sum_{i=1}^n \left(g_i f(x_i) + \frac{1}{2} h_i f^2(x_i) \right) \\
&= \arg\min_f \sum_{i=1}^n \frac{h_i}{2} \left(-g_i/h_i - f(x_i) \right)^2 \\
&= \arg\min_f \sum_{i=1}^n w_i \left(z_i - f(x_i) \right)^2
\end{aligned}$$

where the last line shows that this is equivalent to *weighted least squares* with observation weights $w_i = h_i/2$ and outcome variable $z_i = -g_i/h_i$.

XGBoost uses this approximate loss to build its trees in a straightforward weighted least squares manner.

7 Appendix: L2 Tree Boosting R Code

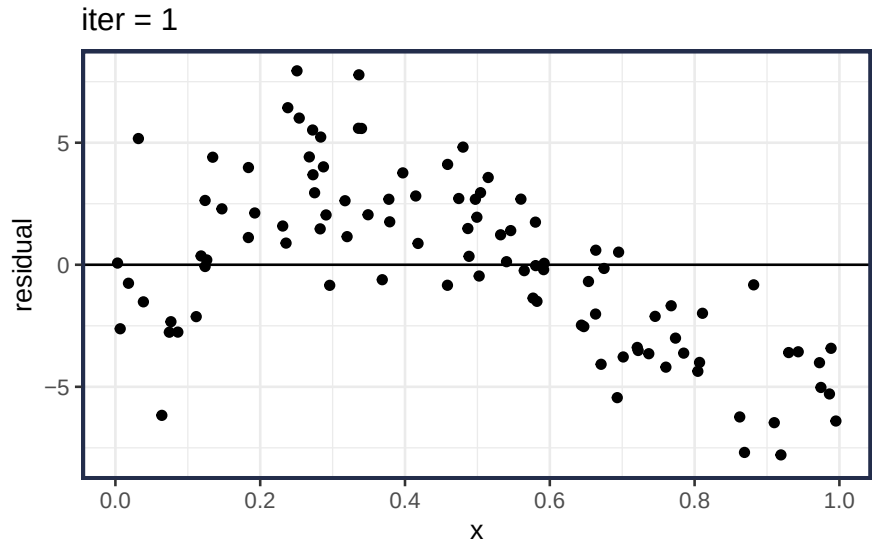
```
#: L2 Boost Algorithm
library(rpart)

# L2boost()
#-------------------------------------------------------------------------------#
# L2 boosted trees (boosted regression trees)
# Inputs:
#  x,y: training data. x should be data frame or matrix, y a vector
#  xtest optional test data (data frame or matrix)
#  M: number of iterations
#  depth: tree depth. depth = 2 gives 4 leaf nodes.
#  nu: shrinkage parameter
# Outputs:
#  YHAT: matrix of in-sample predictions (predicting x)
#  R: matrix of residuals
#  YHAT.test: matrix of predictions for xtest
#  TREE: list of rpart trees
#-------------------------------------------------------------------------------#

L2boost <- function(x, y, xtest=NULL, M=100, depth=2, nu=.1){

  #- use training data if test data is not specified
  if(is.null(xtest)) {
    xtest = x
  }

  #- storage
  n = length(y)
  R = YHAT =  matrix(NA_real_, n, M)
  YHAT_test = matrix(NA_real_, nrow(xtest), M)
  colnames(YHAT) = colnames(YHAT_test) = colnames(R) = paste0("iter = ", 1:M)
  TREE = vector("list", M)
  names(TREE) = paste0("iter = ", 1:M)

  #-- 1) initialize model with mean
  mu = mean(y)
  yhat = rep(mu, nrow(x))
  yhat_test = rep(mu, nrow(xtest))

  for(m in 1:M){

    #-- 2a) Calculate Residuals
    r = y - yhat
    R[,m] = r

    #-- 2b) Fit regression tree
    tree = rpart(r ~ ., data = x,
                 maxdepth = depth,   # control tree depth
                 cp = -1,            # no pruning
                 minsplit = 0,       # allow all splits
                 minbucket = 1,      # no minimum on leaf size
                 method = "anova",   # least-squares loss function
                 xval = 0)           # no cross-validation

    TREE[[m]] = tree

    #-- 2c) Update model
    yhat = yhat + nu*predict(tree, x)
```

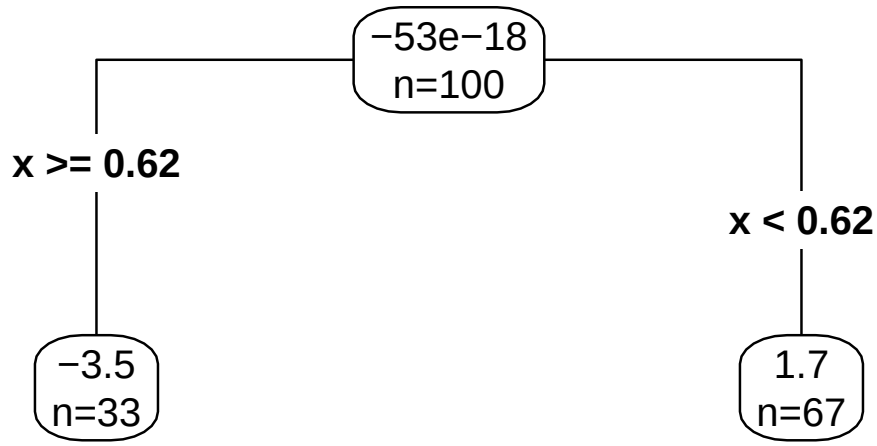
```
    YHAT[, m] = yhat

    yhat_test = yhat_test + nu*predict(tree, xtest)
    YHAT_test[, m] = yhat_test

  }

  #-- 3) Output
  return(list(YHAT=YHAT, R=R, YHAT.test=YHAT_test, TREE=TREE))
}
```

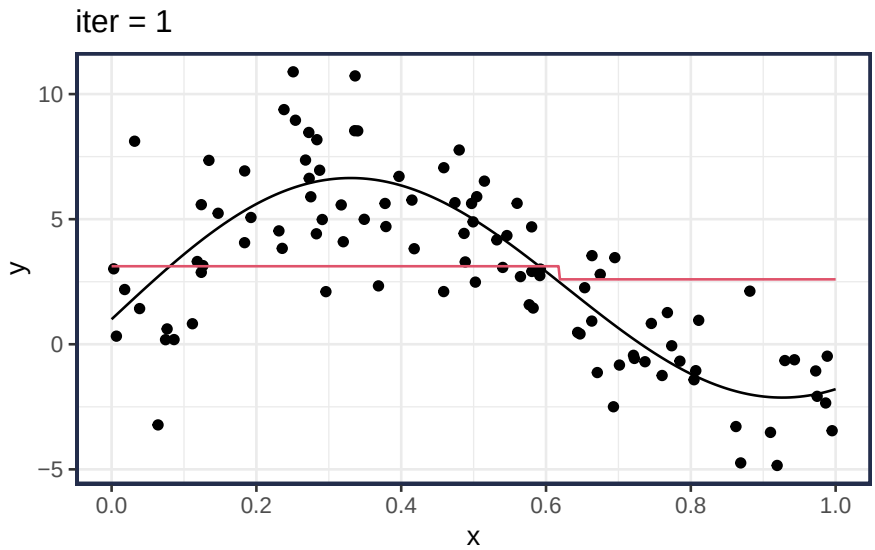
```
#: Data Generation
n = 100                                   # number of observations
generate_x <- function(n) runif(n)        # U[0,1]
f <- function(x) 1 + 2*x + 5*sin(5*x)     # true mean function
sd = 2                                    # stdev for error

set.seed(825)                             # set seed for reproducibility
x = generate_x(n)                         # get x values
y = f(x) + rnorm(n, sd=sd)                # get y values
data_train = data.frame(x, y)             # training data
x_eval = seq(0, 1, length=500)            # evaluation points
```

```
#: Implement L2 boosting
L2 = L2boost(data.frame(x), y, xtest=data.frame(x=x_eval),  # data
             depth = 1, M = 100, nu = .1)              # tuning parameters
```

```
#: Plotting
library(tidyverse)   # for ggplot2 package
library(rpart.plot)  # for prp()

# set iteration
i = 1

# Residual Plot
ggplot(data_train, aes(x)) +
  geom_point(aes(y = L2$R[,i]), col="black") +
  geom_hline(yintercept=0, col="black")  +
  scale_x_continuous(breaks=seq(0, 1, by=.20)) +
  coord_cartesian(ylim=c(-8, 8)) +
  labs(y="residual", title=colnames(L2$R)[i])

# Tree
prp(L2$TREE[[i]], type=4, extra=1, branch=1, clip.right.labs = FALSE, roundint=FALSE)

# Model prediction
ggplot(data_train, aes(x, y)) +
  geom_point() +
  annotate("line", x=x_eval, y=f(x_eval), color = "black") +
  geom_line(data=tibble(x=x_eval, y=L2$YHAT.test[,i]), col=2) +
  scale_x_continuous(breaks=seq(0, 1, by=.20)) +
  labs(title=colnames(L2$R)[i])
```







8 Appendix: xgboost and lightgbm

```r
#: Load required packages
library(tidyverse)
library(palmerpenguins) # data
library(xgboost)
library(lightgbm)
```

We will use the palmer penguins data:

```r
palmerpenguins::penguins
#> # A tibble: 344 x 8
#>   species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>              <dbl>         <dbl>             <int>       <int>
#> 1 Adelie  Torgersen           39.1          18.7               181        3750
#> 2 Adelie  Torgersen           39.5          17.4               186        3800
#> 3 Adelie  Torgersen           40.3          18                 195        3250
#> 4 Adelie  Torgersen           NA            NA                  NA          NA
#> 5 Adelie  Torgersen           36.7          19.3               193        3450
#> 6 Adelie  Torgersen           39.3          20.6               190        3650
#> # i 338 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

with a goal of predicting the `species` from the other variables. This is a three-class probability estimation problem. Most tree-based models can handle multiple classes naturally.

Note that the `sex` and `island` predictors are categorical.

8.1 xgboost

Requires a numeric matrix as input, and suggests using a special xgb formatted matrix for efficiency. The approach here is to create a *recipe* that does some pre-processing to handle missing values and categorical predictors and then create the special matrix.

Notes:

- The recipe removes observations with missing outcomes. It also sets unknown categorical/nominal values to an explicit level. Any missing values in the resulting data are handled internally by xgboost.
- the categorical (or nominal) predictors are dummy encoded.
- the three level outcome variables *species* is recoded to be $\{0, 1, 2\}$. All xgboost data must be numeric, even if it represents a nominal variable.

```r
library(recipes) # part of tidymodels
library(xgboost)

#: pre-processing specs
rec_xgb = recipe(species ~ ., data = palmerpenguins::penguins) %>%
  # Handle missing data
  step_naomit(species) %>%  # remove cases with missing outcome label
  step_unknown(all_nominal_predictors()) %>% # add "unknown" level if missing
  # replace categorical/factors with numeric
  step_dummy(all_nominal_predictors(), one_hot = FALSE) %>%
  step_integer(species, zero_based = TRUE) %>%
  # train or prepare the pre-process step using the data provided in recipe()
  prep()
```

```r
#: xgboost data object
xgb_data =
  xgb.DMatrix(
    data = bake(rec_xgb, new_data = NULL, composition = "matrix",
                all_predictors()),
    label = bake(rec_xgb, new_data = NULL, all_outcomes()) %>% pull()
  )

#: tuning parameters
xgb_tuning = list(
  learning_rate = .1,
  gamma = 1,
  max_depth = 2,
  subsample = .80,
  objective = "multi:softprob", # multi-class loss function
  num_class = 3  # this is required for multi-class problems
)
# Note, the number of iterations, nrounds, is not part of the params.

#: fit model
set.seed(1234) # xgboost can use stochastic resampling
xgb = xgb.train(data = xgb_data,
              params = xgb_tuning, nrounds = 100, # tuning parameters
              verbose = 0) # don't print output during training

#: make predictions
predict(xgb, xgb_data) %>% head()
#>        [,1]     [,2]     [,3]
#> [1,] 0.9934 0.003587 0.003052
#> [2,] 0.9933 0.003597 0.003060
#> [3,] 0.9933 0.003597 0.003060
#> [4,] 0.5629 0.019531 0.417574
#> [5,] 0.9934 0.003587 0.003052
#> [6,] 0.9934 0.003587 0.003052
```

8.2 lightgbm

Following a similar process to what is required for xgboost. LightGBM requires a matrix as input, and suggests using a special formatted matrix. The approach here follows the recipe that is used for xgboost with the exception that lightgbm can internally handle categorical predictors.

Notes:

- The recipe removes observations with missing outcomes. It also sets unknown categorical/nominal values to an explicit level. Any missing values in the resulting data are handled internally by lightgbm.
- the categorical (or nominal) predictors are integer encoded $\{0, 1, ...\}$ so they are numeric, and then handled internally by lightGBM. The `categorical_features` argument in the `lgb.Dataset()` function is used to specify which predictors should be treated as categorical.
- the three level outcome variables *species* is recoded to be $\{0, 1, 2\}$. All lightgbm data must be numeric, even if it represents a nominal variable.

```r
library(recipes) # part of tidymodels
library(lightgbm)

#: pre-processing specs
```

```r
rec_lgbm = recipe(species ~ ., data = palmerpenguins::penguins) %>%
  # Handle missing data
  step_naomit(species) %>%  # remove cases with missing outcome label
  step_unknown(all_nominal_predictors()) %>%        # add "unknown" level if missing
  # replace categorical/factors with numeric
  step_integer(all_nominal_predictors(), zero_based = TRUE) %>%
  step_integer(species, zero_based = TRUE) %>%
  # train or prepare the pre-process step using the data provided in recipe()
  prep()

#: lgb data object
X = bake(rec_lgbm, new_data = NULL, composition = "matrix", all_predictors())
lgbm_data =
  lgb.Dataset(
    data = X,
    colnames = colnames(X),
    categorical_feature = c("island", "sex"),
    label = bake(rec_lgbm, new_data = NULL, all_outcomes()) %>% pull()
  )

#: tuning parameters
lgbm_tuning = list(
  # settings
  objective = "multiclass",
  num_class = 3,
  # tuning parameters
  nrounds = 100,
  min_gain_to_split = 1,
  learning_rate = 0.1,
  max_depth = 2,
  bagging_freq = 1,        # enable subsampling
  bagging_fraction = 0.8, # subsample size
  bagging_seed = 123, # controls the internal sampling
  seed = 987          # controls feature sampling
)

#: fit model
lgbm = lightgbm(data = lgbm_data, params = lgbm_tuning, verbose = -1)

#: make predictions
predict(lgbm, X) %>% head()
#>        [,1]     [,2]     [,3]
#> [1,] 0.9979 0.001190 0.000868
#> [2,] 0.9973 0.001540 0.001123
#> [3,] 0.9972 0.001700 0.001123
#> [4,] 0.8176 0.015829 0.166561
#> [5,] 0.9976 0.001413 0.001031
#> [6,] 0.9979 0.001190 0.000868
```

8.3 Tidymodels

The `bonsai` and `xgboost` packages are necessary to use lightgbm and xgboost in `parsnip`.

```r
library(tidymodels)
library(bonsai)  # for lightgbm
library(xgboost) # for xgboost
```

This defines the `parsnip` model specification for xgboost, modifies the recipe to leave the outcome as a factor, and adds it to a *workflow*.

```r
#: pre-processing specs
rec_xgb = recipe(species ~ ., data = palmerpenguins::penguins) %>%
  # Handle missing data
  step_naomit(species) %>%  # remove cases with missing outcome label
  step_unknown(all_nominal_predictors()) %>% # add "unknown" level if missing
  # replace categorical/factors with numeric
  step_dummy(all_nominal_predictors(), one_hot = FALSE)

# Define the XGBoost model specification
xgb_spec = boost_tree(
  trees = 100,                    # nrounds
  tree_depth = 2,                 # Max depth of the trees
  learn_rate = 0.1,               # Learning rate
  loss_reduction = 1,             # Gamma (min_split_loss)
  sample_size = 0.8,              # Subsample ratio
  mode = "classification"         # For multi-class classification
) %>%
  set_engine("xgboost", objective = "multi:softprob", num_class = 3)

#: Create XGBoost workflow (combine recipe with model specification)
xgb_wf = workflow(preprocessor = rec_xgb, spec = xgb_spec)


# Fit the model
set.seed(1234)
xgb_fit = fit(xgb_wf, data = palmerpenguins::penguins)

# Make predictions on the training data
predict(xgb_fit, palmerpenguins::penguins, type = "prob") %>% head()
#> # A tibble: 6 x 3
#>   .pred_Adelie .pred_Chinstrap .pred_Gentoo
#>          <dbl>           <dbl>        <dbl>
#> 1        0.994         0.00315      0.00273
#> 2        0.994         0.00315      0.00273
#> 3        0.994         0.00315      0.00273
#> 4        0.552         0.0216       0.426
#> 5        0.994         0.00315      0.00273
#> 6        0.994         0.00315      0.00273
```

```r
#: pre-processing specs
rec_lgbm = recipe(species ~ ., data = palmerpenguins::penguins) %>%
  # Handle missing data
  step_naomit(species) %>%  # remove cases with missing outcome label
  step_unknown(all_nominal_predictors()) # add "unknown" level if missing


# Define the lightgbm model specification
lgbm_spec = boost_tree(
  trees = 100,                    # nrounds
  tree_depth = 2,                 # Max depth of the trees
  learn_rate = 0.1,               # Learning rate
  loss_reduction = 1,             # min_gain_to_split
  sample_size = 0.8,              # Subsample ratio
  mode = "classification"         # For multi-class classification
) %>%
  set_engine("lightgbm", objective = "multiclass", num_class = 3, num_threads = 6) %>%
  set_args(
    bagging_seed = 123  # controls the internal sampling
  )

#: Create XGBoost workflow (combine recipe with model specification)
```

```r
lgbm_wf = workflow(preprocessor = rec_lgbm, spec = lgbm_spec)


# Fit the model
lgbm_fit = fit(lgbm_wf, data = palmerpenguins::penguins)

# Make predictions on the training data
predict(lgbm_fit, palmerpenguins::penguins, type = "prob") %>% head()
#> # A tibble: 6 x 3
#>   .pred_Adelie .pred_Chinstrap .pred_Gentoo
#>          <dbl>           <dbl>        <dbl>
#> 1        0.998         0.00119     0.000868
#> 2        0.997         0.00154     0.00112
#> 3        0.997         0.00170     0.00112
#> 4        0.818         0.0158      0.167
#> 5        0.998         0.00141     0.00103
#> 6        0.998         0.00119     0.000868
```